

EXAM-READY STUDY NOTES

Introduction to Software Engineering

Skills required to transform requirements into working software

Coverage	Lectures 01 to 10: Introduction; Software Quality; Application Domains; Software Engineering Process; Software Engineering Principles; Compiler Case Study; Software Engineering and Other Disciplines
Grounding	Ghezzi, Jazayeri and Mandrioli, <i>Fundamentals of Software Engineering</i> ; Pressman, <i>Software Engineering: A Practitioner's Approach</i> ; Pfleeger and Atlee, <i>Software Engineering: Theory and Practice</i> ; Hamlet and Maybee, <i>The Engineering of Software</i> ; Sommerville, <i>Software Engineering</i>
Structure	Nine chapters following the slide order. Each chapter ends with a "Key Points to Remember" summary; Chapter 9 provides consolidated revision aids.

Table of Contents

1. Introduction to Software Engineering	3
1.1 Course Context and Primary References	3
1.2 The Scale and Significance of the Software Industry	3
1.3 Software Development: Resources, Skillset, Process and Lifecycle	3
1.4 Challenges Facing the Software Industry	3
1.5 Illustrative Programming Examples: Why Engineering Matters	4
1.6 Why Software Engineering Is Needed	5
1.7 Problem Solving: Analysis and Synthesis	5
1.8 Defining Software and Software Engineering	5
2. The Software Engineering Team and the Software Lifecycle	6
2.1 Stakeholders: Who Does Software Engineering?	6
2.2 Composition of the Development (SE) Team	7
2.3 The Software Lifecycle (Detailed Phases)	7
2.4 Software Engineers: Approach and Responsibilities	8
3. Defects, Software Quality, Costs and the Nature of Software	8
3.1 Terminology for Describing Bugs	8
3.2 Defects and Software Quality	8
3.3 Cost Associated with Defect Correction	9
3.4 Costs in Software Engineering	9
3.5 Software Engineering Challenges: "Good Enough" Software	9
3.6 The Nature of Software	10
3.7 Software Evolution and Lehman's Laws	10
3.8 Worked Examples of Evolving Requirements	11
4. Qualities of Software and Quality Models	12
4.1 What Is a Good Software Product?	12
4.2 Classifications of Software Qualities (Ghezzi, Chapter 2)	12
4.3 Software Quality Aspects (Product Qualities)	12
4.4 Process Qualities	15
4.5 Software Quality Models	15
5. Software Application Domains and the Software Engineering Process	17
5.1 Software Application Domains	17
5.2 Legacy Software	17
5.3 Application Domains and Relevant Quality Characteristics	18
5.4 Class Activities: Identifying Relevant Quality Aspects	18
5.5 The Software Engineering Process	19
5.6 Software Engineering Practice	20
6. Software Engineering Principles	21
6.1 Principles, Methods, Techniques, Methodologies and Tools	21
6.2 Rigor and Formality	21
6.3 Separation of Concerns	22
6.4 Modularity	22
6.5 Coupling	23
6.6 Cohesion	24
6.7 The Modularity Trade-off	24
6.8 Separation of Concerns and Modularity Revisited	25
7. Software Engineering Principles (Continued)	26
7.1 Anticipation of Change	26
7.2 Generality	27
7.3 Incrementality	28
7.4 Abstraction	28
7.5 Hamlet's Practical Guidelines	29
8. Case Study and Software Engineering in Relation to Other Disciplines	30
8.1 Case Study: SE Principles and Qualities in a Compiler	30
8.2 Software Engineering and Other Disciplines	31
9. Consolidated Revision Aids	33
9.1 Frequently Confused Terms	33
9.2 Lists to Memorise	33
9.3 Typical Examination Questions	34

1. Introduction to Software Engineering

1.1 Course Context and Primary References

The course is subtitled "*Skills required to transform requirements into working software.*" Its aim is to move the student from writing programs to engineering software: producing systems that satisfy stated requirements, within cost and schedule, and that remain usable and changeable over time.

Item	Detail
Assessment (tentative)	Quizzes (announced and unannounced) 10%; Assignments, Project and Class Exercises 20%; Mid-Term Exams 30%; Final Exam 40%
Grading	Absolute grading. Passing requires both the minimum attendance and the minimum marks requirements to be met.
Principal textbooks	Ghezzi, Jazayeri and Mandrioli, <i>Fundamentals of Software Engineering</i> (2nd ed.); Hamlet and Maybee, <i>The Engineering of Software: Technical Foundations for the Individual</i> ; Pressman, <i>Software Engineering: A Practitioner's Approach</i> (9th ed.)
Additional references	Pfleeger and Atlee, <i>Software Engineering: Theory and Practice</i>

1.2 The Scale and Significance of the Software Industry

Software is a major economic sector. The lecture cites the following indicators (Gartner, 2022), which the slides also express in Bitcoin-equivalent terms to convey magnitude:

- Size of the software industry: in excess of **USD 600 billion**.
- Global IT spending: approximately **USD 4.2 trillion**.

Because so much economic value depends on software, failures in how software is produced carry large financial consequences. This is the economic motivation for a disciplined engineering approach.

1.3 Software Development: Resources, Skillset, Process and Lifecycle

Software development draws on three interdependent elements:

- **Resources:** people, time, budget, hardware and tools.
- **Skillset:** the technical and managerial competencies of the team.
- **Process:** the ordered set of activities through which software is produced.

When the process is applied to building a software product, its sequence of phases is called the **software lifecycle**. The typical phases, in order, are:

Requirements Analysis	Design	Coding	Testing	Deployment	Maintenance
-----------------------	--------	--------	---------	------------	-------------

1.4 Challenges Facing the Software Industry

Industry studies (Standish Group) report that only about **33%** of software projects are fully successful, that is, delivered on time, within budget and with the required functionality. The factors that affect the project success rate are:

1. Project size and complexity.
2. Customer satisfaction level.
3. Development and budget process.
4. Skills of developers and managers.

Jones (2008) shows that these factors are associated with **defects**. Defects are therefore the common thread linking project problems to project outcomes.

1.4.1 Causal Chain: Industry Challenges and Software Defects

The lecture presents a causal model (Kan 2002; Galin 2004; Jones 2008). It can be reproduced in an examination as the following chain:

1. **Root causes:** customer-developer communication failure; faulty definition of requirements; documentation, design and coding errors.
2. These root causes **may result in defects**.
3. The factors affecting the project success rate **cause** defects.
4. Defects **impact** (a) **cost and schedule** and (b) the **project success rate**.
5. Defects **indicate** the **customer satisfaction level** and **indicate software quality**. Customer satisfaction, in turn, impacts the project success rate.
6. Governing principle: **the lower the defects, the higher the software quality**.

1.5 Illustrative Programming Examples: Why Engineering Matters

The lecture presents two small C++ programs twice: first with meaningless identifiers (`arr`, `a`, `b`) and then with meaningful identifiers. The programs are functionally identical, yet only the second version is readily understandable. This demonstrates that software quality is not limited to producing correct output; **understandability** and **maintainability** depend on how the code is written.

1.5.1 Example A: Computing the Mode

Version with meaningful names (abridged):

```
int data[10]      = {1,5,5,5,5,4,4,4,2,3};
int elements[5]   = {1,2,3,4,5};
int frequencies[5] = {0,0,0,0,0};
int mode = 0, indexOfMostFrequentElement = 0;
for (int i = 0; i < 10; i++)
    for (int j = 0; j < 5; j++)
        if (data[i] == elements[j]) frequencies[j] = frequencies[j] + 1;
for (int i = 0; i < 5; i++)
    if (frequencies[i] > indexOfMostFrequentElement)
        indexOfMostFrequentElement = i;
mode = indexOfMostFrequentElement + 1;
```

Analytical observation. Renaming the variables exposes a latent **fault**: the condition compares a *frequency* with an *index*. For this particular input the program happens to print the correct mode (5), so the fault does not cause a **failure** here. With other inputs it would. This illustrates two points examined later: good naming improves the ability to detect faults, and a fault does not necessarily produce a failure (Section 3.1). A correct design would store the maximum frequency and its index separately.

1.5.2 Example B: Finding the Second-Largest Element

The meaningful version uses `largest`, `secondLargest` and `temp`. The loop scans the array. When an element exceeds `secondLargest`, the element is assigned to it, and the two variables are swapped if it also exceeds `largest`. For the input `{1,2,3,3,4,6,7,1,2,1}` it outputs 6. Two points follow. First, with meaningless names (`a`, `b`, `c`) the intent cannot be recovered without tracing. Second, the requirement itself is ambiguous: should duplicates of the maximum count as the second largest? This is a **requirements-definition** issue, not a coding issue.

1.5.3 Exercise: Sorting Without Nested Loops

Problem. Ten unique non-negative integers in the range 0 to 20 are stored in an array. Sort them in ascending order without nested loops. Additional memory may be used.

Standard solution (direct addressing / counting sort).

1. Create a Boolean array `present[21]`, initialised to false.
2. In a single loop over the input, set `present[value] = true`.

3. In a second, separate loop over indices 0 to 20, append each index k for which `present[k]` is true to the output array.

This solution runs in $O(n + k)$ time with $O(k)$ extra space. It works because the stated constraints (uniqueness and a small, known range) are exploited. The exercise shows that analysing requirements and constraints carefully produces better engineering solutions.

1.6 Why Software Engineering Is Needed

Software is embedded throughout modern life: toasters, microwave ovens, air-conditioner temperature control, routers, surgical equipment, search engines, chatbots, aircraft and air-defence systems. Such software makes lives more comfortable, efficient and effective. **Software engineering practices ensure the economic development of good software that improves living standards.** Studying the discipline enables the practitioner to deliver software of known quality, predict and control cost and schedule, work effectively in teams, and manage change over a product's lifetime.

1.7 Problem Solving: Analysis and Synthesis

Engineering problem solving proceeds in two complementary directions:

- **Analysis:** decomposing a large *problem* into smaller, more manageable *subproblems* (Subproblem 1 ... Subproblem n) until each can be understood and solved.
- **Synthesis:** composing the individual *solutions* (Solution 1 ... Solution n) into a single integrated *solution* to the original problem.

Analysis is top-down, from problem to parts. Synthesis is bottom-up, from partial solutions to the whole. Both underpin the principles of separation of concerns and modularity (Chapter 6).

1.8 Defining Software and Software Engineering

1.8.1 What Is Software?

Software

Computer programs together with the associated configuration data and documentation needed for the programs to be operated correctly (consistent with Sommerville's definition).

- **Configuration data** set up the programs for a particular environment or use.
- **System documentation** describes the structure of the system and supports developers and maintainers.
- **User documentation** explains how to use the system.

Example (Farmers and Gardens). Three farmers each tend three gardens, and each garden yields a different fruit (Farmer 1: Gardens 1-3; Farmer 2: Gardens 4-6; Farmer 3: Gardens 7-9). A program that allocates fruit picking is software only when it comes with its configuration data (e.g., the farmer-garden-fruit matrix) and documentation describing how it is structured and used.

1.8.2 What Is Engineering?

The job of an engineer is to:

1. Make things work.
2. Apply theories, methodologies and tools appropriately.
3. Provide solutions even when no applicable theory or method exists.
4. Work within financial and organisational constraints.

1.8.3 What Is Software Engineering?

The lecture refines the definition in stages, each more complete than the last:

1. Solving problems that involve computing and computers.
2. Solving such problems in a *consistent and systematic* manner.

3. A set of *guidelines* for solving computing-related problems systematically.
4. **Engineering software while remaining concerned with all aspects of software production.**

Parnas's classic characterisation, cited by Ghezzi et al., describes software engineering as the "**multi-person construction of multi-version software.**" *Multi-person* refers to team, communication and management issues. *Multi-version* refers to evolution and maintenance.

Software Engineering (course definition)

A disciplined, consistent and systematic effort to construct (design and build) and maintain software in a timely and cost-effective manner.

The IEEE standard glossary gives a complementary formal definition: the application of a *systematic, disciplined, quantifiable* approach to the development, operation and maintenance of software.

1.8.4 Aspects of Software Production

Software engineering is concerned with all of the following:

1. The **size** of the software system to be built.
2. The **complexity** of the system.
3. The **need for, and involvement of, teams.**
4. The **technical process** of developing software.
5. **Management activities**, such as managing the project and the teams.
6. The development of **tools, theories and methods** that support software production.

Key Points to Remember

- Software industry exceeds USD 600 billion; global IT spend is about USD 4.2 trillion (Gartner 2022).
- Only about 33% of projects succeed (Standish Group). The four success factors are size/complexity, customer satisfaction, development and budget process, and developer/manager skills. All are associated with defects (Jones 2008).
- Lower defects mean higher quality. Defects affect cost, schedule and success rate, and they indicate customer satisfaction and quality.
- Software = programs + configuration data + documentation (system and user).
- Engineer's job: make things work, apply theory and tools, solve problems without theory, respect financial and organisational constraints.
- SE = disciplined, consistent, systematic effort to construct (design + build) and maintain software in a timely, cost-effective manner. Parnas: "multi-person construction of multi-version software."
- Analysis decomposes a problem; synthesis composes solutions.
- Six aspects of software production: size, complexity, teams, technical process, management, and tools/theories/methods.

2. The Software Engineering Team and the Software Lifecycle

2.1 Stakeholders: Who Does Software Engineering?

A **stakeholder** is any party with an interest in the outcome of a software project. The lecture identifies three principal participants and their relationships:

Participant	Role	Relationship
Customer	Sponsors (pays for) system development	Enters a contractual obligation with the software engineer and expresses needs

Participant	Role	Relationship
User	Uses the system	Has needs that the delivered software system must satisfy
Software Engineer (Developer)	Builds the system	Receives money and needs from the customer; delivers the software system to the user

The customer and the user may be the same person or different parties. For example, a hospital management may purchase a system that nurses use. Treating them as distinct helps prevent requirements that satisfy the sponsor but not the actual users.

2.2 Composition of the Development (SE) Team

Following Pfleeger and Atlee, a development team contains the following roles:

Role	Responsibility
Requirements analysts	Work with customers to identify and document the requirements.
Designers	Produce a system-level description of what the system is supposed to do, and how.
Programmers	Write lines of code to implement the design.
Testers	Catch faults.
Trainers	Show users how to use the system.
Maintenance team	Fix faults that show up after delivery.
Librarians	Prepare and store documents, such as software requirements.
Configuration management team	Maintain correspondence among the various artefacts (requirements, design, code, tests) and control their versions.

2.2.1 Mapping of Roles to Lifecycle Steps (Who Does What?)

Pfleeger's "roles of the development team" figure maps roles onto lifecycle steps. The roles overlap, so one person may hold several roles:

- **Analyst:** requirements analysis and definition; system design.
- **Designer:** system design; program design.
- **Programmer:** program design; program implementation; unit testing; integration testing.
- **Tester:** unit testing; integration testing; system testing.
- **Trainer:** system delivery; maintenance.

2.3 The Software Lifecycle (Detailed Phases)

The lifecycle phases, as given in the course, are:

1. **Requirements analysis and definition:** establish what the system must do (functional requirements) and the constraints on it (non-functional requirements), and record them in a specification.
2. **System (architecture) design:** define the overall structure, including major components, their responsibilities and interfaces.
3. **Program (detailed/procedural) design:** specify the internal logic, algorithms and data structures of each component.
4. **Writing programs (coding/implementation):** translate the detailed design into source code.
5. **Testing:** at three levels. *Unit* testing checks individual components. *Integration* testing checks interacting components. *System* testing checks the complete system against its requirements.
6. **System delivery (deployment):** install the system in its operational environment and train users.
7. **Maintenance:** correct, adapt and enhance the system after delivery.

2.4 Software Engineers: Approach and Responsibilities

- Software engineers adopt a **systematic and organised approach**, applied effectively, to produce **high-quality software**.
- They **may have to use ad hoc approaches**, because some real, complex problems cannot be solved with the elegant theories of computer science. New algorithms or pragmatic solutions may be required.
- The ultimate purpose is to **improve living standards**.

Key Points to Remember

- Three principal stakeholders: customer (sponsors), user (uses), software engineer (builds).
- Eight team roles: requirements analysts, designers, programmers, testers, trainers, maintenance team, librarians, configuration management team.
- Seven lifecycle phases: requirements analysis and definition, system (architecture) design, program (detailed) design, coding, testing (unit, integration, system), delivery, maintenance.
- Engineers are systematic, but may need ad hoc approaches where theory is insufficient.

3. Defects, Software Quality, Costs and the Nature of Software

3.1 Terminology for Describing Bugs

The informal word "bug" is replaced by precise terms (Pfleeger; consistent with IEEE usage):

Error	A mistake made by a human while performing a software activity (e.g., misunderstanding a requirement).
Fault	The result of an error: an incorrect step, definition or statement in a software artefact (requirements, design or code). A fault is also called a <i>defect</i> and exists in the static product.
Failure	A departure of the system's observed behaviour from its required behaviour. It is observed at run time.

Relationship: Human **error** leads to a **fault**, which may lead to a **failure**. A fault causes a failure only when the faulty part is executed under conditions that expose it. Therefore a system may contain faults yet exhibit no failures during a given period of use (see Example A in Section 1.5.1). A single failure can also be caused by several faults.

3.2 Defects and Software Quality

Defect	A failure to meet the requirements (or expectations) of the user or customer (Kan 2002).
---------------	--

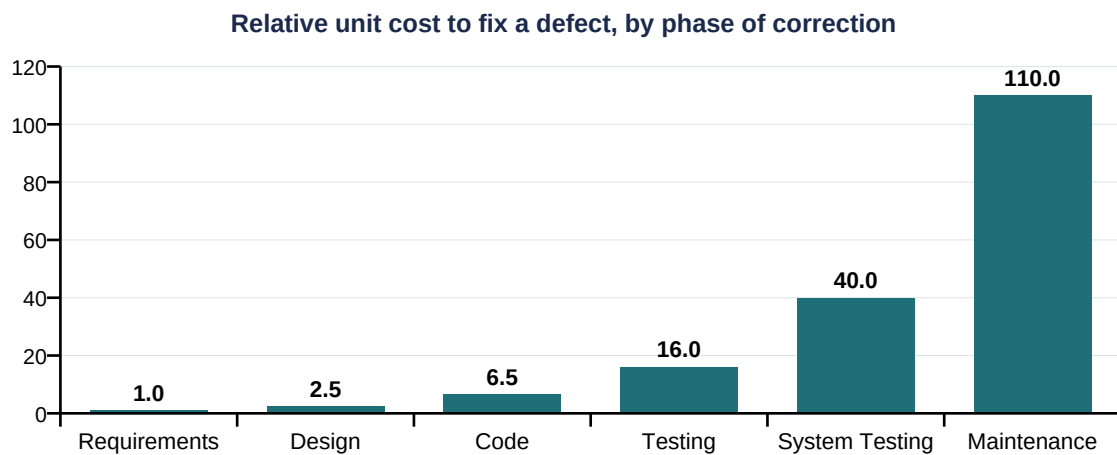
There is no single definition of software quality. The classical definitions are:

Definition of Quality	Source
Fitness for use	Juran and Gryna (1970)
Conformance to requirements	Crosby (1979)
Freedom from deficiencies	Juran (1988)
Lack of defects	Kan (2002)

Key economic principle: removing defects increases quality, but it also **incurs cost**. Quality and cost must therefore be balanced.

3.3 Cost Associated with Defect Correction

The relative unit cost of fixing a defect rises sharply the later the defect is found (Khoshgoftaar et al. 1996; Galin 2004):



Source: Khoshgoftaar et al. (1996); Galin (2004). "System testing" denotes testing before and at delivery.

Phase in which the defect is corrected	Relative unit cost	Multiple of requirements cost
Requirements	1.0	1x
Design	2.5	2.5x
Code	6.5	6.5x
Testing	16.0	16x
System testing (before and at delivery)	40.0	40x
Maintenance (operations)	110.0	110x

Interpretation. A defect that costs one unit to fix during requirements costs about 110 units in operation. The cost rises because a late fix requires rework of every downstream artefact (design, code, tests, documentation), plus redeployment and possible business damage. Hence **defect prevention and early detection reduce cost**. Examples include requirements reviews and design inspections.

3.4 Costs in Software Engineering

The principal cost categories are:

1. Cost to **change** software.
2. Cost to **overcome a mistake** (defect).
3. Cost of **developing** software.
4. **Operational** cost.
5. **Deployment** cost.

3.5 Software Engineering Challenges: "Good Enough" Software

Software engineers aim for **good enough software**, which must satisfy three sets of demands at once:

Demand	Constituents
Acceptable quality	Usability, security, reliability, performance
Cost effectiveness	Engineering and operational feasibility; limited development budget

Demand	Constituents
Timely completion	Limited time

These demands frequently **conflict**. For example, higher reliability requires more testing, which consumes budget and time. The engineer's task is to **manage these conflicts** through explicit, justified trade-offs.

3.6 The Nature of Software

Software differs fundamentally from physical engineering products. The lecture draws several comparisons:

- **Software vs. an engine (wear-out):** an engine physically wears out. Software does not wear out, but it does deteriorate as it is changed.
- **Software vs. a house:** a house is rarely restructured after construction. Software is routinely modified after delivery.
- **Software vs. cars (models and versions):** car models are manufactured in identical copies. Software exists in many versions, and "manufacture" (copying) costs almost nothing. The cost lies in development.
- **Stability, problems and defects:** software requirements and environments are rarely stable, which is a continual source of defects.

Two properties summarise the nature of software (Ghezzi):

- **Malleable:** software is easy to modify, which invites frequent and sometimes undisciplined change.
- **Human intensive:** software is an intellectual product created by people, not machines. Its quality depends on human skill and communication.

Pressman adds that software is *developed or engineered*, not *manufactured* in the classical sense. Software *does not wear out*. Much software is still *custom-built*, although component-based reuse is increasing.

3.6.1 Failure Curve for Hardware (the "Bathtub" Curve)

The failure rate of hardware plotted against time has three regions:

1. **Infant mortality:** a high initial failure rate caused by design or manufacturing defects. It falls as these are corrected.
2. **Steady state:** a low, roughly constant failure rate over the useful life.
3. **Wear-out:** a rising failure rate as components deteriorate through dust, vibration, temperature extremes and other environmental effects.

3.6.2 Failure Curve for Software

1. **Idealised curve:** a high initial failure rate due to undiscovered defects. It falls as defects are fixed and then flattens to a steady level. There is **no wear-out phase**, because software is not affected by environmental maladies.
2. **Actual curve (Pressman):** during its life, software undergoes change. Each change may introduce new defects, which causes **spikes** in the failure rate. Before the curve returns to its steady level, another change occurs. The minimum failure rate therefore slowly **rises**. **Software deteriorates because of change, not wear.**

Aspect	Hardware	Software
Shape	Bathtub curve	Idealised: falls, then flat; Actual: spikes with rising baseline
Cause of late failures	Physical wear-out	Side effects of change (deterioration)
Remedy for failure	Replace worn part with a spare	Modify design or code; no spare part exists

3.7 Software Evolution and Lehman's Laws

Software that is used in a real-world environment must evolve. Lehman's laws of software evolution describe this behaviour. The four laws covered in the course are:

Year	Law	Statement
1974	Continuing Change	A system must be continually adapted, or it becomes progressively less satisfactory. This continues until it becomes more economical to replace it with a new or restructured version.
1974	Increasing Complexity (Entropy)	The complexity (entropy) of a system increases with time, unless work is done to maintain or reduce it.
1991	Continuing Growth	The functional content of a system must be continually increased to maintain user satisfaction over its lifetime.
1996	Declining Quality	The quality of a system will appear to decline unless it is rigorously maintained and adapted to changes in its operational environment.

For completeness, Lehman's full set also includes self-regulation, conservation of organisational stability, conservation of familiarity and the feedback system law. The four above are the ones examined in this course.

3.8 Worked Examples of Evolving Requirements

Farmers and gardens. Each farmer must pick an equal number of fruits from gardens that yield different quantities (e.g., Farmer 1: 9, 5, 1; Farmer 2: 6, 7, 2; Farmer 3: 1, 5, 9). The simple statement hides several questions. Equal across farmers or across fruits? What happens when totals differ? Such ambiguity is the kind of faulty requirements definition identified in Section 1.4.1.

Demand forecasting. A client requests a time-series model to forecast demand. After the first version, new requirements arrive: *write the forecast output to a file*, and *run once for multiple businesses*. This demonstrates Lehman's law of continuing change in miniature. A design that anticipated such change (e.g., separating output handling and parameterising the business) evolves at low cost. A rigid design does not.

Key Points to Remember

- Error (human mistake) leads to a fault/defect (in the artefact), which may lead to a failure (observed wrong behaviour). A fault need not cause a failure.
- Defect = failure to meet user or customer requirements or expectations (Kan 2002).
- Quality definitions: fitness for use (Juran and Gryna 1970); conformance to requirements (Crosby 1979); freedom from deficiencies (Juran 1988); lack of defects (Kan 2002).
- Relative defect-fix cost: 1 (requirements), 2.5 (design), 6.5 (code), 16 (testing), 40 (system testing), 110 (maintenance). Prevention is cheapest.
- Five cost types: change, defect correction, development, operation, deployment.
- Good-enough software balances acceptable quality, cost-effectiveness and timely completion. These conflict and must be managed.
- Software is malleable and human-intensive. It does not wear out but deteriorates through change (actual failure curve with spikes). Hardware follows the bathtub curve.
- Lehman: Continuing Change (1974), Increasing Complexity (1974), Continuing Growth (1991), Declining Quality (1996).

4. Qualities of Software and Quality Models

4.1 What Is a Good Software Product?

Good software engineering must always include a **strategy for producing quality software**. Product quality is not a single attribute. It has **multiple facets**, and different stakeholders judge it by different criteria:

- **Users** judge **external characteristics**, such as correct functionality and the number and type of failures.
- **Designers and maintainers** judge **internal characteristics**, such as ease of modification.

4.2 Classifications of Software Qualities (Ghezzi, Chapter 2)

4.2.1 External vs. Internal Qualities

Type	Description
External	Visible to the users of the system (e.g., reliability, usability, performance).
Internal	Concern the developers of the system and deal with the <i>structure</i> of the software (e.g., modularity, understandability of code).

The two types are linked. Internal qualities often *help achieve* external ones. For instance, a well-structured, verifiable design (internal) supports reliability (external).

4.2.2 Product vs. Process Qualities

- **Product**: the software delivered to the customer. Ghezzi notes that the "product" also includes the **work products** (intermediate artefacts) created during development, such as requirements specifications, designs, test plans, user manuals and code. Different work products matter to different stakeholders.
- **Process**: the software production process used to build the product. Process quality strongly influences product quality. For example, a process with systematic reviews tends to yield more reliable software.

4.3 Software Quality Aspects (Product Qualities)

4.3.1 Correctness

Correctness	A mathematical property that establishes the equivalence between the software and its specification . A program is correct if it behaves exactly as its functional specification requires.
--------------------	--

- Correctness is **absolute**: any deviation from the specification makes the software incorrect, however minor the deviation.
- It presupposes a precise specification. Without one, correctness cannot be established.
- It can be pursued **analytically** (verification and testing) or **constructively**, for example by using **standard proven libraries and modules**, established **methodologies** and disciplined **processes**.

4.3.2 Reliability (Dependability)

Reliability	The probability that the software will operate as expected over a specified time interval. It is sometimes termed dependability .
--------------------	---

- Reliability is **relative**. Software with a fault that rarely causes failures may still be highly reliable.
- Ghezzi's relationship: the set of **correct** programs is a subset of the set of **reliable** programs. A correct program is reliable (with respect to its specification), but a reliable program need not be correct.

4.3.3 Robustness

Robustness

The software **behaves reasonably** even in circumstances not anticipated by the specification, such as **unexpected input** or abnormal situations (e.g., hardware malfunction).

- Robustness concerns what the specification does *not* state. If a situation is specified, handling it becomes a matter of correctness. If it is not specified, handling it is a matter of robustness.
- Example: a program that prints a helpful error message and continues after invalid input, instead of crashing.

4.3.4 Performance

Performance

The amount of work the software performs **per unit of time**, e.g., transactions per second or response time.

Performance affects usability and scalability. It is assessed by **measurement** (monitoring), **analysis** (complexity or queueing models) and **simulation**.

4.3.5 Efficiency

Efficiency

How **economically** the software uses computing resources, such as processor time, memory, storage and network bandwidth.

In Ghezzi's treatment, performance is essentially equated with efficiency. The course distinguishes *work done per unit time* (performance) from *economy of resource use* (efficiency).

4.3.6 Usability (User Friendliness)

Usability

How easily human users can learn and use the software, also called **user friendliness**.

Usability is subjective and depends on the user population. Novices and experts value different interfaces. It is influenced by the user interface, consistency, documentation and configurability.

4.3.7 Verifiability

Verifiability

The degree to which the **properties of the software** (e.g., correctness, performance) **can be verified** easily.

Verifiability is usually an internal quality. It is improved by modular design, disciplined coding practices and a suitable programming language. It can also be an external requirement, for example in safety-critical systems.

4.3.8 Maintainability

Maintainability

The ease with which software can be modified after its initial release.

Maintenance consumes the majority of total software cost. Ghezzi, citing Lientz and Swanson, distinguishes three categories:

Category	Approx. share	Purpose
Corrective	about 20%	Removing residual faults that remain in the delivered product.
Adaptive	about 20%	Adjusting the software to changes in its environment (new hardware, operating system, database, regulations).
Perfective	over 50%	Changing the software to improve its qualities or to add or modify functions requested by users.

Maintainability decomposes into two sub-qualities:

- **Repairability:** can defects be corrected in a **reasonable amount of time with a reasonable amount of work**? It relates chiefly to corrective maintenance. It is strongly supported by **modularity**, because faults are confined to fewer modules.
- **Evolvability:** can the software be changed **easily and economically** to satisfy a new or changed requirement? It relates to adaptive and perfective maintenance. It depends on **modularity**, **up-to-date documentation**, a **feasibility study** of the change and an **impact analysis** identifying what the change affects. Evolvability tends to decline with each release unless it is actively preserved, which is consistent with Lehman's laws.

4.3.9 Reusability

Reusability

The extent to which a product, or parts of it, can be used, possibly with minor changes, to build another product.

Reuse applies at many levels: code components and libraries, designs (patterns), requirements and even people's expertise. Reuse reduces cost and improves reliability, because reused components have already been validated.

4.3.10 Portability

Portability

The ability of software to **run in different environments**, such as different hardware platforms, operating systems or browsers.

Portability is achieved by isolating environment-dependent parts behind well-defined interfaces.

4.3.11 Understandability

Understandability

For tasks of inherently similar difficulty, the degree to which the software system is **easy to understand**.

Understandability is internal: it supports evolvability and verifiability. Externally, understandable behaviour means the system behaves predictably for the user.

Example: control structures and understandability. Unstructured code with `goto` statements makes control "skip around" among statements, which makes the logic very hard to follow:

```

    benefit = minimum;
    if (age < 75) goto A;      // may be a function call
    benefit = maximum;
    goto C;
    if (AGE < 65) goto B;
    if (AGE < 55) goto C;
A:   if (AGE < 65) goto B;
    benefit = benefit * 1.5 + bonus;
    goto C;
B:   if (age < 55) goto C;
    benefit = benefit * 1.5;
C:   next statement

```

Restructured into a single selection structure, the same intent is immediately clear:

```

if      (age < 55) benefit = minimum;
else if (AGE < 65) benefit = minimum + bonus;
else if (AGE < 75) benefit = minimum * 1.5 + bonus;
else      benefit = maximum;

```

The transformation is called **restructuring**. Note also that the original code contains **unreachable statements** (the two lines after the first `goto C`) and inconsistent identifiers (`age/AGE`). Both are symptoms of poor understandability.

4.3.12 Interoperability

Interoperability	The ability of a system to coexist and cooperate with other systems, for example exchanging data through standard formats or interfaces.
-------------------------	---

4.3.13 Security

Security	The ability to keep information safe , to avoid intrusions , and to have a low number of vulnerabilities .
-----------------	---

Security is commonly described through **Confidentiality** (only authorised parties can access information), **Integrity** (information cannot be altered without authorisation) and Availability (the CIA triad).

4.4 Process Qualities

Quality	Definition
Productivity	The amount of work done by a software team per unit of time. It denotes the efficiency of the process . It is difficult to measure; metrics such as lines of code or function points each have limitations.
Timeliness	The ability to deliver a product on time . A late product may lose its market value even if its quality is high. Incremental delivery supports timeliness.
Visibility	The degree to which all steps of the process and their current status are clearly documented . Visibility enables management to assess progress and make decisions. It is also called transparency.

4.5 Software Quality Models

A quality model organises quality characteristics into a structured hierarchy so that quality can be specified, measured and evaluated. Pressman (9th ed., Chapter 15) discusses the models below.

4.5.1 McCall's Quality Model (1977)

McCall's model organises eleven quality factors around three aspects of a software product, drawn as the three sides of a triangle:

Aspect (triangle side)	Concern	Quality factors
Product Operation	Operational characteristics	Correctness, Reliability, Usability, Integrity, Efficiency
Product Revision	Ability to undergo change	Maintainability, Flexibility, Testability
Product Transition	Adaptability to new environments	Portability, Reusability, Interoperability

Each factor is linked to lower-level, measurable **criteria (metrics)**, for example traceability, completeness, consistency, accuracy, error tolerance, execution efficiency, storage efficiency, access control, access audit, operability, training, communicativeness, simplicity, conciseness, instrumentation, self-descriptiveness, expandability, generality, modularity, software-system independence, machine independence, communications commonality and data commonality. One criterion may support several factors, which is shown in the model as a many-to-many mapping.

4.5.2 FURPS (Hewlett-Packard)

FURPS was developed at Hewlett-Packard (Grady). It groups quality attributes into five categories:

Category	Meaning
Functionality	Feature set and capabilities, generality of functions, and security of the overall system.
Usability	Human factors, aesthetics, consistency and documentation.
Reliability	Frequency and severity of failure, accuracy of output, mean time to failure, recoverability and predictability.

Category	Meaning
Performance	Processing speed, response time, resource consumption, throughput and efficiency.
Supportability	The ease with which a system can be installed and its problems localised . It encompasses maintainability (extensibility, adaptability, serviceability), testability, compatibility and configurability.

The slide's question "Maintainability?" is answered by the model itself: in FURPS, maintainability is subsumed under **Supportability**.

4.5.3 ISO/IEC 25010 Quality Model

ISO/IEC 25010 is part of the SQuaRE (Systems and software Quality Requirements and Evaluation) series. The course cites version 1 in 2011 and version 2 in 2017. It defines two complementary models:

- **Quality in Use:** characteristics appropriate when the software is **used in a particular context**, namely effectiveness, efficiency, satisfaction, freedom from risk and context coverage.
- **Product Quality:** characteristics that focus on the **static and dynamic nature** of the software system. The 2011 model lists eight characteristics:

Product quality characteristic	Representative sub-characteristics
Functional suitability	Functional completeness, correctness, appropriateness
Performance efficiency	Time behaviour, resource utilisation, capacity
Compatibility	Co-existence, interoperability
Usability	Appropriateness recognisability, learnability, operability, user error protection, user interface aesthetics, accessibility
Reliability	Maturity, availability, fault tolerance, recoverability
Security	Confidentiality, integrity, non-repudiation, accountability, authenticity
Maintainability	Modularity, reusability, analysability, modifiability, testability
Portability	Adaptability, installability, replaceability

Note: a later revision (2023) adds *safety* as a characteristic and renames some items. For examination purposes, follow the version taught in class.

4.5.4 Comparison of Quality Models

Feature	McCall	FURPS	ISO/IEC 25010
Origin	McCall et al., 1977 (US Air Force)	Hewlett-Packard (Grady)	International standard, 2011
Structure	3 aspects, 11 factors, criteria	5 categories	Quality in Use + 8 product characteristics
Maintainability	Under Product Revision	Under Supportability	Explicit characteristic
Distinctive feature	Links factors to measurable criteria	Simple, practical checklist	Context-of-use view; widely adopted

Key Points to Remember

- External qualities are visible to users. Internal qualities concern developers and structure. Product vs. process; the product includes intermediate work products.
- Correctness is absolute (equivalence with the specification). Reliability is relative and probabilistic. Correct programs are a subset of reliable programs. Robustness means reasonable behaviour in unspecified situations.
- Performance = work per unit time. Efficiency = economical resource use.
- Maintenance shares: corrective about 20%, adaptive about 20%, perfective over 50%. Maintainability = repairability + evolvability.

- Evolvability depends on modularity, updated documentation, feasibility study and impact analysis.
- Restructuring `goto` code into structured selection improves understandability.
- Process qualities: productivity, timeliness, visibility.
- McCall: Operation (correctness, reliability, usability, integrity, efficiency); Revision (maintainability, flexibility, testability); Transition (portability, reusability, interoperability).
- FURPS: Functionality, Usability, Reliability, Performance, Supportability (includes maintainability).
- ISO 25010: Quality in Use (context) and Product Quality (static and dynamic).

5. Software Application Domains and the Software Engineering Process

5.1 Software Application Domains

Pressman classifies software into broad categories. Each category places different demands on the engineering process and emphasises different qualities.

Domain	Description	Examples
System software	Programs written to service other programs.	Compilers, editors, operating-system components
Application software	Stand-alone programs that solve a specific business need.	Information systems such as a leave management system or a point-of-sale system
Engineering / scientific software	Programs that simulate or analyse natural and engineered phenomena.	Astronomy, volcanology, molecular biology
Real-time systems	Must respond to events within a predefined and strict period of time. Response time is quantitatively specifiable and verifiable, so a late response is an incorrect response (<i>response time as correctness</i>).	Flight control, factory fault detection
Embedded systems	Reside within a product or system and perform special functions for the user and for the system itself.	Temperature measurement in a control shed; keypad control in a microwave oven
Product-line software	Software for a group of related products marketed under a single brand name.	Microsoft Office (Word, PowerPoint, Excel)
Web / mobile applications	Browser-based and mobile-device-based applications.	Online banking portal, mobile ride-hailing app
Artificial intelligence software	Uses heuristics-based algorithms to solve complex problems.	Robotics software, expert systems for disease diagnosis, forecasting systems

5.2 Legacy Software

Legacy software	Older systems that have been continually modified to meet changes in requirements and platforms, and that remain in use.
------------------------	---

Characteristics (problems):

- Difficult and costly to change further.
- Inextensible designs.
- Convolutional code.
- Poor or non-existent documentation.

- Unarchived results of test cases.
- Poorly managed change history.
- Risky to evolve.

Why they persist: legacy systems **perform core functions** and are **indispensable to the business**, so they cannot simply be discarded. Pressman advises that a legacy system need not be changed if it meets its users' needs and runs reliably. It must evolve when it must be adapted to new environments, enhanced for new business requirements, extended to interoperate with modern systems, or re-architected for new computing environments.

5.3 Application Domains and Relevant Quality Characteristics

Domain	System characteristics	Most relevant quality characteristics
Information systems	Data-oriented; storage and retrieval of data. E.g., banking, library and HRM systems.	Data integrity, security, data availability, transaction performance
Real-time systems	Control- or event-oriented; respond to events within a predefined, strict time; sensor-based. E.g., flight control, automatic fault detection in a factory, surveillance systems.	Correctness (including response time), reliability, safety (expressed as negative requirements, i.e., what the system must never do)
Distributed systems	Computers connected through a communication network. E.g., distributed file systems.	Portability, interoperability, performance, reliability
Embedded systems	Hardware-specific; the software interfaces with other system components. E.g., software inside ovens and dishwashers.	Efficiency, robustness

General rule: quality characteristics related to **human-computer aspects** (e.g., usability) are relevant in **all** domains.

5.4 Class Activities: Identifying Relevant Quality Aspects

For each scenario, the examinable skill is to state whether a quality is relevant and how it could be measured. Model answers:

Quality	Sorting n unique integers	Farmers and gardens	POS system (small grocery)
Correctness	Relevant. Output is a permutation in ascending order; verify with test oracles or proof.	Relevant. Allocation satisfies the "equal fruits" rule; compare against hand-computed cases.	Relevant. Totals, taxes and stock updates match the specification; use test cases.
Efficiency	Relevant. Time complexity (comparisons or swaps) and memory use.	Relevant for large inputs; count operations and memory.	Relevant. CPU and memory on low-cost hardware.
Robustness	Relevant. Behaviour with empty, duplicate, negative or huge input.	Relevant. Behaviour with missing gardens or impossible equalisation.	Relevant. Invalid barcodes, power loss mid-transaction, network loss.
Usability	Limited (library routine).	Relevant if used by farmers; measure task time and error rate.	Highly relevant; cashier training time, errors per transaction, satisfaction surveys.
Evolvability	Low to moderate.	Relevant (more farmers, fruits or rules); measure effort per change.	Relevant (new payment methods, tax rules); measure change effort and impact.
Performance	Implied by efficiency.	Moderate.	Relevant; checkout response time, transactions per minute.
Reliability	Implied by correctness.	Moderate.	Relevant; mean time between failures, failure rate.
Verifiability	High (easily tested).	Moderate.	Relevant; test coverage, auditability of transactions.

5.5 The Software Engineering Process

Process

A series of steps involving **activities, constraints and resources** that produce an intended output of some kind (Pfleeger).

Everyday processes include preparing for exams, conducting a software competition, organising a trip and writing a term project report. A process typically involves a set of **tools and techniques**, for example **block diagrams** and **notations**.

5.5.1 Why Is a Process Useful?

1. It imposes **consistency and structure** on a set of activities.
2. It guides us to **understand, control, examine and improve** the activities.
3. It enables us to **capture experience and pass it along** to others.

5.5.2 Framework Activities (Pressman)

A generic process framework consists of five framework activities, applied in every project regardless of size or domain:

Activity	Purpose
Communication	Communicate and collaborate with the customer and other stakeholders to understand objectives and gather requirements.
Planning	Create a "map" (roadmap or project plan) defining tasks, risks, resources, work products and schedule.
Modeling	Build models to understand the requirements better and to provide a design that satisfies them.
Construction	Code generation (manual or automated) and the testing needed to uncover errors.
Deployment	Delivery of the software (complete or as an increment) to the customer , who evaluates it and provides feedback .

5.5.3 Process Flow

Process flow describes how the framework activities, and the actions and tasks within each, are organised in sequence and time (Pressman):

1. **Linear process flow**: the five activities execute in sequence, once each, from communication to deployment.
2. **Iterative process flow**: one or more activities are repeated before proceeding to the next (e.g., returning from modeling to communication).
3. **Evolutionary process flow**: the activities execute in a **circular** manner. Each circuit produces a more complete version of the software, called an **increment released**.
4. **Parallel process flow**: one or more activities execute in parallel with others (e.g., modeling one aspect while constructing another).

5.5.4 The Waterfall Model

The waterfall model is the classical **linear, sequential** lifecycle model. Each phase is completed and its output approved before the next phase begins, like water cascading down steps. The course diagram (after Pfleeger) shows these stages:

1. Requirements analysis
2. System design
3. Program design
4. Coding
5. Unit and integration testing
6. System testing
7. Acceptance testing

8. Operation and maintenance

Advantages	Limitations
Simple, easy to understand and manage	Assumes requirements are stable and fully known at the start
Clear milestones and documentation at each phase	Working software appears late; customer feedback comes late
Suits well-understood, stable projects	Defects found late are expensive to fix (see Section 3.3)
Imposes discipline on development	Poorly suited to change, which is inevitable (Lehman)

5.5.5 Software Lifecycle as a Process

When a process involves building a software product, the process may be referred to as the **software (product) lifecycle**. Its phases are requirements analysis and definition, system (architecture) design, program (detailed/procedural) design, writing programs (coding/implementation), testing (unit, integration, system), system delivery (deployment), and maintenance (see Section 2.3).

5.5.6 Umbrella Activities

Umbrella activities are applied **throughout** the process, surrounding and supporting the framework activities. They help the team manage and control progress, quality, change and risk. The course identifies five:

Abbr.	Activity	Purpose
PTC	Project Tracking and Control	Assess progress against the plan and take action to maintain the schedule.
RM	Risk Management	Identify and assess risks that may affect the outcome or quality, and plan mitigation.
SQA	Software Quality Assurance	Define and conduct the activities needed to ensure software quality.
TR	Technical Reviews	Assess work products to uncover and remove errors before they propagate to the next activity.
CM	(Software) Configuration Management	Manage the effects of change throughout the process, including versions and baselines.

Pressman's full list also includes **measurement**, **reusability management** and **work product preparation and production**.

5.6 Software Engineering Practice

Pressman states that the essence of software engineering practice follows Polya's problem-solving method. Each step maps to engineering activities:

Generic step	SE activity
1. Understand the problem	Communication and analysis
2. Plan a solution	Modeling and software design
3. Carry out the plan	Code generation
4. Examine the results for accuracy	Testing and quality assurance

Key Points to Remember

- Eight application domains: system, application, engineering/scientific, real-time, embedded, product-line, web/mobile, AI.
- Real-time: response time is part of correctness. Embedded: resides in a product and serves the user and itself.
- Legacy software is costly and risky to change, poorly documented and convoluted, but performs core, indispensable functions.

- Domain qualities: IS (integrity, security, availability, transaction performance); real-time (correctness, reliability, safety); distributed (portability, interoperability, performance, reliability); embedded (efficiency, robustness). Human-computer qualities apply everywhere.
- Process = steps involving activities, constraints and resources to produce an intended output. Benefits: consistency, understanding and control, capturing experience.
- Framework activities: Communication, Planning, Modeling, Construction, Deployment.
- Process flows: linear, iterative, evolutionary, parallel.
- Umbrella activities: PTC, RM, SQA, TR, CM.
- Practice: understand, plan, carry out, examine.

6. Software Engineering Principles

6.1 Principles, Methods, Techniques, Methodologies and Tools

Ghezzi (Chapter 3) presents a layered view of the software development process:

1. **Principles** (innermost, most general and stable): general, abstract statements of desirable properties of the process and product.
2. **Methods and techniques**: *methods* are general guidelines governing an activity; *techniques* are more technical and mechanical approaches, often with restricted applicability.
3. **Methodologies**: packaged combinations of methods and techniques that promote a common approach to problem solving.
4. **Tools** (outermost, most specific and changeable): software that supports the application of techniques, methods and methodologies.

Principles form the foundation. Tools change quickly, whereas principles remain valid. Ghezzi's seven principles are **rigor and formality, separation of concerns, modularity, abstraction, anticipation of change, generality and incrementality**. Lectures 7-8 cover the first three (this chapter). Lecture 9 covers the remaining four, together with Hamlet's practical guidelines (Chapter 7).

6.2 Rigor and Formality

Rigor

Precision and exactness in carrying out software engineering activities.

Rigor enables engineers to:

- **Repeatedly** produce reliable products.
- **Control the costs** of the products.
- **Increase confidence** in the reliability of the products.

Formality

The **highest degree of rigor**: the use of **mathematical laws** to derive and evaluate the software and its process (e.g., formal specification, mathematical proof of correctness).

Rigor does not stifle creativity. It complements creativity by making results precise and checkable. Formality is not always required; the appropriate degree of rigor depends on the criticality of the software.

Beneficial effects of rigor and formality apply to:

Quality	How rigor helps
Reliability	Precise specifications and systematic verification reduce faults.
Verifiability	Precisely stated properties can be checked or proved.
Maintainability	Precise documentation makes changes safer.
Reusability	Precisely specified components can be reused with confidence.

Quality	How rigor helps
Portability	Precisely defined environment dependencies can be isolated.
Understandability	Unambiguous descriptions are easier to understand.
Interoperability	Precisely specified interfaces enable cooperation between systems.

6.3 Separation of Concerns

Separation of concerns	The principle of dealing with different aspects of a problem separately , so that each can be concentrated on individually and complexity is mastered.
-------------------------------	---

Concerns can be separated along several dimensions (Ghezzi Chapter 3; Hamlet Chapter 1):

Dimension	Explanation and example
Time	Activities are scheduled in different periods, which is the basis of lifecycle phases (requirements analysis, design, coding, testing, deployment, maintenance).
Qualities	Different qualities are addressed separately, e.g., first make the program correct, then restructure it for efficiency.
Views	The system is analysed from different perspectives, e.g., a data-flow view and a control-flow view.
Parts (size)	The system is divided into components that are dealt with individually. This dimension leads directly to modularity .
Management vs. technical	Managerial concerns (schedule, cost, staffing) are separated from technical concerns (design, coding).
Problem domain vs. implementation domain	What the problem requires (e.g., business rules) is separated from how it is realised (e.g., data structures, languages).

Caution: separating concerns may cause some global optimisations to be missed. The benefit is that decisions become manageable, which usually outweighs this cost.

6.4 Modularity

Modularity	The principle of dividing a complex system into simpler pieces called modules , keeping unrelated aspects of the system separate so that each aspect can be studied in isolation. It is separation of concerns applied to parts.
-------------------	--

If the principle is applied well, **each module has a single purpose** (the **single responsibility principle**) and is **relatively independent** of the others. The benefits are:

- Each module is easier to **understand and develop**.
- It is **easier to locate faults**, because there are fewer suspect modules per fault.
- It is **easier to change** the system, because a change to one module affects relatively few others.

Ghezzi identifies the capabilities that modularity provides:

- **Decomposability:** dividing a system top-down into modules (analysis).
- **Composability:** building a system bottom-up from existing modules (synthesis).
- Ghezzi additionally lists the capability to **understand** the system in terms of its pieces and the capability to **modify** it by changing only a few pieces.

6.4.1 Example: Modular Decomposition of a Quiz System

Subsystem	Modules
Student subsystem	Submission Management (Submission Verification, Submission Storage); Quiz Conduction and Display

Subsystem	Modules
Faculty subsystem	User Management; Quiz Management (Quiz Generation, Question Management); Report Management (Grading, Attempt Reporting)

The Quiz System decomposes into two subsystems, each subsystem into modules, and some modules into sub-modules. This illustrates decomposability. Integrating these modules into the complete system illustrates composability.

The degree to which a design separates concerns is measured by two concepts of module independence: **coupling** and **cohesion**.

6.5 Coupling

Coupling	The degree of interdependence between modules . High coupling is not desired . The goal is low (loose) coupling.
-----------------	--

Level	Description
Tightly coupled	Modules depend a great deal on each other (many dependencies).
Loosely coupled	Modules have some dependence, but their interconnections are weak (some dependencies).
Uncoupled	Modules have no interconnections at all and are completely unrelated (no dependencies).

6.5.1 Types of Coupling

The following types are listed from **highest (worst)** to **lowest (best)** for the classical scale (Pfleeger). They are followed by the additional types described by Pressman.

- 1. Content coupling (worst).** One component **modifies an internal data item** of another component, or one component **branches into the middle of another**. For example, Module B executes `goto D1`, where D1 is a label inside Module D. Setting a private data member of another module is also content coupling. It destroys information hiding.
- 2. Common coupling.** Components depend on **common (global) data**. It becomes unclear which component is responsible for setting a particular value of the common variable. Evaluating a change requires **tracing back to all components that access those data**. *Example:* several modules read and write a global variable `taxRate`; if one module sets it wrongly, every other module is affected, and finding the culprit is hard.
- 3. Control coupling.** One component **passes parameters to control the activity** of another. The controlled component cannot function without direction from the controlling component. **Remedy:** make each component perform only **one function**. *Example:*

```
bool act(int x) {
    if (x == 0) return false;
    else      return true;
}
void call() {
    act(1); // passes a value that controls act()'s flow
}
```

- 4. Stamp coupling.** **Complex data structures** are passed between modules. The interface is more complex, because the modules must **agree on the data's format and organisation**, even if only part of the structure is used.
- 5. Data coupling (best form of interaction).** Only **data values**, not structured data, are passed. It is simpler and less likely to be affected by changes in data representation. It is the **easiest to trace** and the easiest to change.

Memory aid (worst to best): "Can Cats Catch Small Ducks?", i.e., Content, Common, Control, Stamp, Data.

Additional types (Pressman):

- Routine call coupling:** one operation invokes another. It is common and often necessary, but it increases connectedness.

- **Type-use coupling:** component A uses a data type defined in component B, e.g., a class member of another class's type. If the type definition changes, every component using it must change.
- **Inclusion / import coupling:** component A imports or includes a package or the content of component B.
- **External coupling:** a component communicates or collaborates with infrastructure components (e.g., operating-system functions, databases, telecommunication functions).

```
class A { /* ... */ };
class B {
    A var;                // data member of type A -> type-use coupling
    void calculate(A data) { // argument of type A -> stamp coupling
        /* ... */
    }
};
```

6.6 Cohesion

Cohesion

A measure of the degree to which the **internal elements of a module** (data, functions, etc.) **are related**, i.e., the degree to which all elements contribute to performing a **single task** assigned to the module. It reflects the internal strength and independence of a module. **High cohesion is desired; low cohesion is not.**

6.6.1 Types of Cohesion (Lowest to Highest)

#	Type	Description	Simple example
1	Coincidental	Parts are unrelated ; grouped arbitrarily (worst).	A "Utilities" module holding printReport(), computeTax() and resetPassword()
2	Logical	Parts perform similar kinds of functions , selected by a control flag.	readInput(flag) reading from keyboard, file or network depending on flag
3	Temporal	Parts are related by timing ; executed at the same time (e.g., T0, T0+X, T0+2X).	startup() that opens files, clears counters and shows a splash screen
4	Procedural	Parts related by the order in which they must be performed.	checkCard(), then enterPIN(), then showMenu() in one module
5	Communicational	Parts access the same data .	Module that prints and also saves the same student record
6	Sequential	Output of one part is the input to the next.	readMarks() feeds computeGPA(), which feeds formatTranscript()
7	Functional	Every part is essential to one single, well-defined function (best).	computeSquareRoot(x)

Memory aid (worst to best): "Can Lazy Teachers Prepare Cool Study Files?", i.e., Coincidental, Logical, Temporal, Procedural, Communicational, Sequential, Functional.

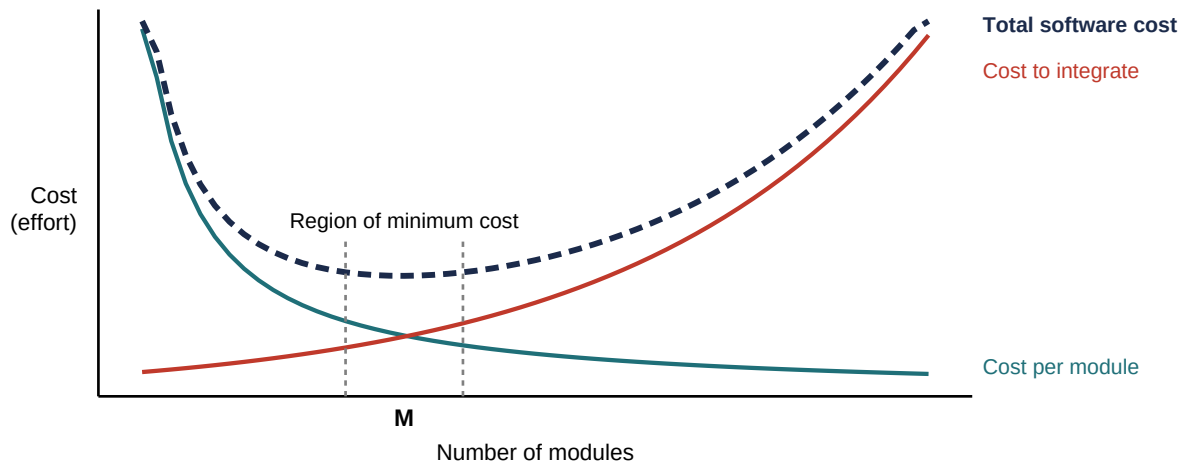
Design goal: low coupling and high cohesion. The two are complementary. A highly cohesive module tends to require fewer interactions with other modules.

6.7 The Modularity Trade-off

Modularity involves a trade-off between **over-modularisation** and **under-modularisation** (Pressman):

1. As the **number of modules** increases, the **cost per module (effort to develop each module)** decreases, because each module becomes smaller and simpler.
2. As the number of modules increases, the **cost to integrate** (interface) the modules **increases**.
3. The **total software cost** is the sum of these two curves. It is U-shaped.
4. There is a **region of minimum cost** at an intermediate number of modules, labelled M. Designers should aim for this region. Too few modules (under-modularisation) produce large, complex modules. Too many

(over-modularisation) produce excessive integration effort.



6.8 Separation of Concerns and Modularity Revisited

- **Intellectual control:** humans can manage complexity only when it is divided into pieces that can be understood one at a time.
- **Independent parts:** parts that can be understood, developed and verified independently allow work to proceed in parallel and keep complexity manageable.
- **The 7 ± 2 rule** (Miller, 1956): human short-term memory holds roughly seven (plus or minus two) items at a time. A module, or a level of decomposition, should therefore present only a limited number of elements. This is a practical guide to sizing modules. (The lecture's word-search puzzle illustrates how difficult it is to find patterns in an unstructured mass of information.)

Key Points to Remember

- Layered view: principles, then methods and techniques, then methodologies, then tools. Principles are the most stable.
- Rigor = precision and exactness. Formality = highest rigor, using mathematical laws. Rigor benefits reliability, verifiability, maintainability, reusability, portability, understandability and interoperability.
- Separation of concerns dimensions: time, qualities, views, parts, management vs. technical, problem vs. implementation domain.
- Modularity: single purpose per module (single responsibility principle); easier understanding, fault location and change; decomposability and composability.
- Coupling (worst to best): content, common, control, stamp, data. Pressman adds routine call, type-use, import/inclusion and external. Low coupling is desired.
- Cohesion (worst to best): coincidental, logical, temporal, procedural, communicational, sequential, functional. High cohesion is desired.
- Total cost = per-module cost (falls) + integration cost (rises). Aim for the minimum-cost region M.
- Intellectual control, independent parts, Miller's 7 ± 2 rule.

7. Software Engineering Principles (Continued)

This chapter completes Ghezzi's seven principles (anticipation of change, generality, incrementality, abstraction). It then covers Hamlet's practical guidelines: identifying customers, consistent terminology, documentation, software as transformation, and reuse.

7.1 Anticipation of Change

Anticipation of change

The principle of identifying, early in development, the changes the software is **likely to undergo**, and designing so that those changes are **easy and cheap** to make. Likely changes are isolated in specific modules (Ghezzi, Ch. 3; Hamlet, Ch. 2).

Why it matters. Lehman's laws (Section 3.7) show that change is unavoidable. A design that ignores it must be rewritten in many places for every new requirement.

Qualities supported:

- **Evolvability:** new or changed requirements are absorbed with little effort.
- **Reusability:** components designed with variation in mind fit other products.

How to apply it (a doable checklist):

1. List what is likely to change: business rules, user interface, hardware or operating system, external services, data formats.
2. Place each likely change behind a stable interface, inside one module.
3. Keep documentation and configuration management up to date, so that versions of the changing parts are controlled.

7.1.1 The Open-Closed Principle (OCP)

Open-Closed Principle

Software entities (classes, modules, functions) should be **open for extension** but **closed for modification** (Meyer; popularised by Martin). New behaviour is added by writing **new code**, not by editing existing, tested code.

Example: a payment processor. Suppose the system first supports credit cards only, and PayPal is requested later.

(a) Without anticipation of change: OCP not followed.

```
class PaymentProcessor {
public:
    void processPayment(const std::string& type, double amount) {
        if (type == "CreditCard")
            std::cout << "Processing credit card payment of $" << amount << "\n";
        // To add PayPal we must MODIFY this class again:
        // else if (type == "PayPal") { ... }
    }
};
```

Every new payment method forces another **if-else** branch inside a class that already works. Each edit risks breaking existing behaviour and requires retesting the whole class.

(b) With anticipation of change: OCP compliant.

```

class PaymentMethod {                                // 1. Open for extension
public:
    virtual ~PaymentMethod() = default;
    virtual void process(double amount) const = 0;
};
class CreditCardPayment : public PaymentMethod {      // 2. Concrete implementations
public:
    void process(double amount) const override {
        std::cout << "Processing credit card payment of $" << amount << "\n"; }
};
class PayPalPayment : public PaymentMethod {
public:
    void process(double amount) const override {
        std::cout << "Processing PayPal payment of $" << amount << "\n"; }
};
class PaymentProcessor {                             // 3. Closed for modification
public:
    void processPayment(const PaymentMethod& method, double amount) {
        method.process(amount);
    }
};

```

To add, say, a wallet payment, a new class **WalletPayment** is written. **PaymentProcessor** is never touched again. The likely change (new payment types) has been anticipated and isolated behind the **PaymentMethod** interface. *(The slide notes that this code was AI-generated; it is used only to illustrate the principle.)*

7.2 Generality

Generality

When asked to solve a specific problem, look for the **more general problem** hidden behind it. A general solution can often be **reused**, and is sometimes even simpler (Ghezzi, Ch. 3).

Example: procedure interfaces in order of increasing generality. The slide says "four" interfaces but shows three (rows 1, 2 and 4 below). Row 3 is the fourth interface from Pfleeger's original version of this example.

#	Interface	Postcondition and comment
1	PROCEDURE SUM: INTEGER	Returns the sum of 3 global variables . Usable only in one context; also causes common coupling.
2	PROCEDURE SUM (a, b, c: INTEGER): INTEGER	Returns the sum of the parameters . Works for any three numbers.
3	PROCEDURE SUM (a[]: INTEGER; len: INTEGER): INTEGER	Returns the sum of the first <i>len</i> elements of array a. Works for lists of any length.
4	PROCEDURE SUM (a[]: INTEGER): INTEGER	Returns the sum of all elements in array a . Most general.

Key technique: parameterising context-specific information. Anything tied to one situation (fixed variables, fixed sizes) is turned into a parameter, so the same routine serves many situations.

Trade-offs of generality:

- A general solution may take **more time and cost** to develop.
- It may be **less efficient** (speed or memory) than a specialised one.
- Excessive generality adds complexity that the product may never need ("over-engineering").

The engineer must judge how general to be, based on the expected reuse and the anticipated changes.

7.3 Incrementality

Incrementality

A process that proceeds in a **stepwise fashion**. The desired goal is reached by successive **increments**, each adding to what was built before (Ghezzi, Ch. 3).

The slide highlights four ideas:

- **Increments:** build and deliver the system in pieces, not all at once.
- **Early subsets:** identify useful subsets of the application that can be built and delivered first.
- **Early feedback:** users evaluate each subset, so misunderstandings are caught early, when they are cheap to fix (Section 3.3).
- **Prototypes:** quick, partial versions built to clarify requirements or test design ideas.

Design structure that enables increments (slide figures).

1. **Design 1 vs. Design 2.** In Design 1, module A uses B, C and D, and the next layer (E, F, G) is used by B and C. The system is layered, so a subset (A with B, C, D) can be built and tested first, and E, F, G added later. In Design 2, A directly uses five modules (B to F), so much more must exist before A can work. Layered designs with limited fan-out suit incremental delivery.
2. **Figures (a), (b), (c): breaking dependency cycles.** (a) A, B and C depend on one another in a cycle, so none can be built or tested alone. (b) B is split into B1 and B2, which gives an acyclic chain (B1 uses A, A uses C, C uses B2). This can be built bottom-up: B2, then C, then A, then B1. (c) A is also split (A1, A2), giving two independent chains. Each chain can be delivered as a separate increment.

Rule to remember: an acyclic, layered "uses" structure allows early subsets. A cyclic one forces everything to be built together.

7.4 Abstraction

Abstraction

A **model or representation that omits some details so that it can focus on other details**. It identifies the important aspects of a phenomenon and ignores the rest (Ghezzi, Ch. 3).

Example: sorting a list L at three levels of abstraction.

Level 1 (most abstract): "Sort L in non-decreasing order." This states *what* is required, with no algorithm.

Level 2: an algorithm outline (selection sort):

```
DO WHILE I is between 1 and (length of L) - 1
  Set LOW to index of smallest value in L(I), ..., L(length of L)
  Interchange L(I) and L(LOW)
ENDDO
```

This adds a great deal of information, but "find the smallest" and "interchange" are still not spelled out.

Level 3 (most detailed): exactly how sorting works:

```
DO WHILE I is between 1 and (length of L) - 1
  Set LOW to current value of I
  DO WHILE J is between I+1 and (length of L)
    IF L(LOW) is greater than L(J) THEN set LOW to current value of J
  ENDIF
ENDDO
Set TEMP to L(LOW)
Set L(LOW) to L(I)
Set L(I) to TEMP
ENDDO
```

Abstraction at every level of computing. The same idea applies to languages. A high-level statement such as `print("Hello World!")` hides the assembly program below it (8086 DOS: load the data segment, load the message address into DX, call interrupt 21H with function 09H to display, then function 4CH to exit). The assembly program in turn hides the **binary machine code** (e.g., 10110100 00001001 ...). Each level lets the programmer think

only about the details that matter at that level.

Benefits: it controls complexity, supports separation of concerns and modularity (a module's interface is an abstraction of its implementation), and allows stepwise refinement from specification to code.

7.5 Hamlet's Practical Guidelines

7.5.1 Identification of "Customers"

Every phase produces work that another phase consumes. Treating the consumer as a **customer** makes each producer responsible for the quality of what they hand over (Hamlet, Ch. 2).

Customer	Is a customer of
Programmer	Designer
Designer	Requirements engineer
Test engineer	All other phases
Programmer	Test engineer (receives fault reports)
User	Maintenance phase

Practical meaning: a designer should write a design that a programmer can actually implement, just as the final product must satisfy the paying customer.

7.5.2 Use of Consistent Terminology

Each term must mean one thing throughout a project.

- **"Specification or design?"** A document describing *what* the system must do is a specification. One describing *how* it will do it is a design. Mixing the words confuses readers about the document's purpose.
- **"Test has failed?"** This is ambiguous. It may mean the software *failed* the test (a failure was found, which is a success for the tester), or that the test itself could not be run properly. Precise terms (error, fault, failure; Section 3.1) remove the ambiguity.

7.5.3 Documentation

- **Record lest you forget it:** decisions not written down are lost.
- **Document assumptions** at each stage, so later phases know what was taken for granted.
- **Traceability:** each requirement can be traced to its design, code and tests, and back.
- **Document structuring:** place content where readers expect to find it.
- **Clarity:** prefer **defining a term once** and using it consistently over repeating varied descriptions.
- **Index:** allow readers to locate topics quickly.

7.5.4 Software as Transformation

View a program as a **transformation from a set of inputs to a set of outputs**. Analyse it in four steps:

1. Identify **all inputs and outputs**.
2. Identify the **valid and invalid ranges** of the inputs and outputs.
3. Identify **special or significant values** of the inputs and outputs.
4. Identify **important combinations** of input and output values.

Worked example: online telephone directory. The directory replaces the printed phonebook. It returns phone numbers for a name, lists area codes for parts of the country, and gives emergency numbers for the user's area.

Step	Application to the directory
Inputs / outputs	Inputs: a name; a region (for area codes); the user's area (for emergency numbers); the type of request. Outputs: phone number(s); area codes; emergency numbers; error messages.

Step	Application to the directory
Valid / invalid ranges	Valid: names made of letters, spaces, hyphens or apostrophes; known regions. Invalid: empty name, digits or symbols in a name, unknown region, unknown area.
Special values	Name not in the directory; several people with the same name; very long names; the user's area missing or not detected.
Important combinations	Name + region (to tell apart people with the same name); request type + missing area (emergency numbers must still be shown, e.g., a national default).

This analysis gives a direct basis for designing **test cases** and for spotting missing requirements.

7.5.5 Reuse

- **Know the existing libraries** before writing new code.
- **Develop the software so that it can be reused**, at the level of both **requirements** and **code**.
- Reusable code needs **well-specified function prototypes** (interfaces): clear names, parameters, return values, pre- and postconditions.

Key Points to Remember

- Anticipation of change: find likely changes early and isolate them; supports evolvability and reusability.
- OCP: open for extension, closed for modification; add new classes behind an interface instead of editing if-else chains.
- Generality: solve the more general problem by parameterising context-specific information. Trade-offs: cost, efficiency, complexity.
- SUM interfaces in increasing generality: globals, then three parameters, then array with length, then whole array.
- Incrementality: increments, early subsets, early feedback, prototypes. Acyclic, layered "uses" structures enable increments; split modules to break cycles.
- Abstraction: a model omitting some details to focus on others. Sorting at three levels; high-level code, then assembly, then machine code.
- Hamlet: identify customers (each phase serves the next); consistent terminology; documentation (record, assumptions, traceability, structure, clarity, index); software as transformation (I/O, ranges, special values, combinations); reuse (know libraries, design for reuse, clear interfaces).

8. Case Study and Software Engineering in Relation to Other Disciplines

8.1 Case Study: SE Principles and Qualities in a Compiler

8.1.1 Components (Phases) of a Compiler

Component	Role
Lexical analyser (scanner)	Reads source code character by character and groups characters into meaningful tokens (keywords, identifiers, operators).
Syntax analyser (parser)	Arranges tokens into a tree (parse tree or abstract syntax tree, AST), compares it with the grammar rules, and reports non-compliances.
Semantic analyser	Checks logical and meaning-based errors, such as type mismatch or use of an undeclared variable. Outputs an updated (annotated) AST.
Intermediate code generator	Produces a machine-independent intermediate representation : the bridge between high-level source code and low-level target code.
Code optimiser	Improves the intermediate code so that the final program runs faster and uses less memory.

Component	Role
Code generator	Translates the optimised intermediate code into the target language (usually assembly or machine language).

8.1.2 Tracing One Statement Through the Compiler

Take the statement $x = y + 5$; where x and y are floating-point variables.

1. **Scanner** produces tokens: identifier x , operator $=$, identifier y , operator $+$, number 5 , symbol $;$.
2. **Parser** builds the tree: `assignment(x, add(y, 5))`.
3. **Semantic analyser** notices that an integer is added to a float and inserts a conversion: `add(y, int_to_float(5))`.
4. **Intermediate code**: `t1 = int_to_float(5); t2 = y + t1; x = t2`
5. **Optimised code**: `t1 = y + 5.0; x = t1` (the conversion is done at compile time, and one temporary is removed).
6. **Target (assembly) code**:

```
LDF  R2, y      ; Load variable y into Register 2
ADDF R2, R2, #5.0 ; Add 5.0 to Register 2
STF  x, R2      ; Store the result from Register 2 into variable x
```

8.1.3 Principles Visible in a Compiler

Principle	How the compiler shows it
Rigor and formality	Tokens are defined by regular expressions and syntax by formal grammars, so the language is precisely specified.
Separation of concerns	Analysis (front end: scanner, parser, semantic analyser) is separated from synthesis (back end: optimiser, code generator).
Modularity	Each phase is a module with one purpose (high cohesion) that passes a well-defined data structure (tokens, AST, intermediate code) to the next (low, data-level coupling).
Abstraction	The intermediate representation hides details of the target machine from the front end.
Anticipation of change / generality	To support a new processor, only the code generator is replaced. To support a new language, only the front end is replaced.
Incrementality	A compiler can be built phase by phase (e.g., scanner first, then parser), each tested before the next is added.

8.1.4 Qualities Relevant to a Compiler

- **Correctness**: the generated code must preserve the meaning of the source program. This is the most critical quality.
- **Reliability**: developers depend on it daily.
- **Robustness**: invalid source code must produce clear error messages, not crashes.
- **Performance and efficiency**: both compilation speed and the speed and memory use of the generated code (the optimiser's goal).
- **Portability and evolvability**: supported by the front end / back end split.
- **Usability**: meaningful, well-located error messages.

8.2 Software Engineering and Other Disciplines

8.2.1 SE and Computer Science

The slide diagram shows how the disciplines relate:

1. **Computer science** contributes **theories** and knowledge of **computer functions**.
2. A real-world **problem** is the other input.

3. **Software engineering** combines the two.
4. The result is **tools and techniques to solve the problem**.

Mutual influence with CS areas:

CS area	Relationship with SE
Programming languages	Languages added features in response to SE needs: modularity (modules, packages, classes) and exception handling (for robustness).
Operating systems	Among the earliest software that was engineered . Their large size exposed the need for SE principles such as layering and abstraction.
Databases	Serve as components of large systems . They provide data abstraction and independence between data and programs.
Artificial intelligence	Supplies techniques (heuristics, learning) used in software and in SE tools; SE practices are needed to build reliable AI systems.
Theoretical CS	Supplies formal models (automata, grammars, logic) that underpin formal specification and verification.

8.2.2 SE and Management Science

Software development is a **multi-person endeavour**, so it depends on management techniques: project estimation, project scheduling, human resource planning, task decomposition, task assignment, project tracking, and hiring and motivating people.

8.2.3 SE and Systems Engineering

- Systems engineering deals with **complex systems** such as chemical plants, factory monitoring systems, flight control systems, buildings and bridges.
- **Software is often a component of a larger system.**
- The software engineer should therefore **take a broader look** and **understand the application area**.
- Typical trade-off: **flexibility vs. performance**. Software makes a system easy to change, while dedicated hardware may be faster.

8.2.4 SE and Other Engineering Fields

SE borrows from, and contributes to, **civil engineering** (design discipline, standards), **mechanical engineering** (embedded control), and **computer (hardware) engineering** (hardware/software co-design). The slide also asks whether **medicine** is related. Software is central to medical devices and diagnosis, where safety and reliability are critical, and SE can learn from medicine's evidence-based practice.

Key Points to Remember

- Compiler phases: scanner, parser (parse tree / AST), semantic analyser, intermediate code generator, optimiser, code generator.
- $x = y + 5$: tokens, tree, type conversion, three-address code, optimised code, assembly (LDF, ADDF, STF).
- Compiler principles: formality (grammars), separation of concerns (front vs. back end), modularity, abstraction (intermediate representation), anticipation of change (retargeting), incrementality.
- CS gives theories and computer functions; together with a problem, SE produces tools and techniques.
- PL (modularity, exception handling), OS (early engineered software), DB (components of large systems), AI, theoretical CS.
- Management science: estimation, scheduling, HR planning, task decomposition and assignment, tracking, hiring and motivating.
- Systems engineering: software is part of a larger system; understand the domain; flexibility vs. performance.

9. Consolidated Revision Aids

9.1 Frequently Confused Terms

Pair	First term	Second term
Error vs. Fault	Human mistake	Its manifestation in an artefact (defect)
Fault vs. Failure	Static flaw in the product	Observed deviation from required behaviour at run time
Correctness vs. Reliability	Absolute; equivalence with the specification	Relative; probability of expected operation over time
Correctness vs. Robustness	Behaviour in specified situations	Reasonable behaviour in unspecified situations
Performance vs. Efficiency	Work done per unit time	Economy of resource use
Repairability vs. Evolvability	Fixing defects in reasonable time and effort	Changing to meet new requirements economically
Product vs. Process quality	Qualities of delivered software and work products	Qualities of the production process (productivity, timeliness, visibility)
External vs. Internal quality	Visible to users	Concern developers; structure of software
Customer vs. User	Sponsors development	Uses the system
Framework vs. Umbrella activities	Core sequence (C, P, M, C, D)	Applied throughout (PTC, RM, SQA, TR, CM)
Coupling vs. Cohesion	Between modules; want low	Within a module; want high
Decomposability vs. Composability	Top-down split into modules	Bottom-up building from modules
Analysis vs. Synthesis	Problem into subproblems	Solutions into overall solution
Abstraction vs. Generality	Hide details to focus on what matters	Solve a broader problem so the solution can be reused
Anticipation of change vs. OCP	General principle: isolate likely changes	Concrete rule: extend with new code, do not modify existing code
Specification vs. Design	What the system must do	How the system will do it

9.2 Lists to Memorise

Topic	Items (in order)
Lifecycle phases	Requirements analysis and definition; system design; program design; coding; testing (unit, integration, system); delivery; maintenance
Waterfall stages	Requirements analysis; system design; program design; coding; unit and integration testing; system testing; acceptance testing; operation and maintenance
Success factors	Size and complexity; customer satisfaction; development and budget process; developer and manager skills
Defect-fix cost	1; 2.5; 6.5; 16; 40; 110
Cost types	Change; defect; development; operational; deployment
Good-enough software	Acceptable quality; cost effectiveness; timely completion
Lehman (course)	Continuing change; increasing complexity; continuing growth; declining quality

Topic	Items (in order)
Maintenance types	Corrective 20%; adaptive 20%; perfective over 50%
FURPS	Functionality; usability; reliability; performance; supportability
McCall aspects	Product operation; product revision; product transition
Framework activities	Communication; planning; modeling; construction; deployment
Process flows	Linear; iterative; evolutionary; parallel
Practice	Understand; plan; carry out; examine
Hamlet guidelines	Identify customers; consistent terminology; documentation; software as transformation; reuse
Transformation steps	All inputs and outputs; valid/invalid ranges; special values; important combinations
Compiler phases	Scanner; parser; semantic analyser; intermediate code generator; optimiser; code generator
Ghezzi principles	Rigor and formality; separation of concerns; modularity; abstraction; anticipation of change; generality; incrementality

9.3 Typical Examination Questions

1. Define software engineering and explain Parnas's phrase "multi-person construction of multi-version software."
2. Distinguish between error, fault and failure with an example. Can a faulty program be reliable? Justify.
3. Explain, with the cost-of-defect data, why early defect detection is economically important.
4. Sketch and compare the failure curves of hardware and software. Why does software deteriorate?
5. State Lehman's laws covered in the course and relate them to evolvability.
6. Differentiate correctness, reliability and robustness, and state the relationship between correct and reliable programs.
7. Describe McCall's model and FURPS. Where does maintainability appear in each?
8. For a given domain (e.g., real-time or information system), identify the most relevant quality characteristics.
9. List Pressman's framework and umbrella activities and explain the four process flows.
10. Explain separation of concerns along its dimensions, and relate it to modularity.
11. Define coupling and cohesion, list their types in order and give an example of each.
12. Explain the modularity trade-off with the cost-versus-number-of-modules graph.
13. State the Open-Closed Principle and show, with a short code example, how it supports anticipation of change.
14. Arrange procedure interfaces in order of generality and discuss the trade-offs of generality.
15. Show three levels of abstraction for sorting a list.
16. Apply "software as transformation" to a given system (e.g., an online telephone directory).
17. List the phases of a compiler and explain which SE principles each design decision illustrates.
18. Explain how SE relates to computer science, management science and systems engineering.